Control Text Adventure Adaption - Prolog Game

Sandy MacDonald

University of Aberdeen, 2026

Abstract

Introduces and adapts a reactive, fault-tolerant exploration, puzzle-solving, combat-adventure focused Prolog reimplementation and loose adaption of the 2019 Action Adventure Control video-game through careful consideration of and design of exhaustive knowledge-base representation facts and intuitive gameplay logic clauses to reimplement such a gameplay-scenario.

Exploration and story is expressed through the use of many location facts, in a labyrinthine and interconnected fashion, where locations can be locked, requiring solution of a puzzle or picking up a unlock object. Furthermore, these locations contain user-readable notes, inner thoughts, to convey story through text plus pickable health kits, health upgrades and weapon charge upgrades to strengthen the player through permanent upgrades.

There are optional puzzles that exist that reward the player with additional lore, health and weapon charge upgrades plus optional player powers such as the ability to telekinetically throw objects at a enemy once per combat scenario, plus the inventory command shows what the player has currently got in their inventory and allows users to place items in their inventory in a room to solve the aforementioned puzzles.

The player can encounter combat when they have picked up the service weapon, enemies can dynamically spawn in locations through the use of the infection system, the players weapon can be augmented through the retrieval of the weapon form, dealing thrice the damage but thrice the weapon charges, plus a optional shield unlock allows players to block a enemies attack once a combat scenario. Health and weapon charge as-well as the enemies hp is given each combat round, critical hits and weapon misfires are also present and defined in my game knowledge-base. There are various enemies, ranging in difficult and a final boss encounter which is significantly harder than all other enemies before finding of the brother.

Tutorialisation is given through use of tips and the help command showcasing every passible command known to the game, plus the players goal of finding her brother is reinforced through use of the quest command and reinforcement through the various notes and inner thoughts the player can optionally read hinting towards how to find the end goal location and the ability to talk to NPC's and gain tips on where and what to go next which can also aid in solving puzzles.

All of this, presented and playable via the prolog interpreter CLI running control.pl or optionally hosted on a web-sever, through running server.pl that opens up a stylised web-page allowing the game to be player in a more user-friendly environment rather than the tricky command-line-interpreter.

Introduction

As extensively described in the abstract, the purpose of the game is controlling the protagonist Jesse, who has recently travelled to the doors of the mysterious Oldest House where something has led her here, to explore both the inside and outside of this building, to find her brother at any cost. Through this exploration, she encounters a surreal world, where mystery is afoot, dialogue is strange and unsettling, her inner thoughts lead her to the discovery of the service weapon, where when picked up combat and infection scenarios can appear, she will now be able to make her way to enemy infested rooms, battling enemies and healing and upgrading her stats until she inevitably finds the boss enemy, defeating said boss drops a key and allows entrance to Dylan's cell where the player can finally talk to her brother and end the game and see through to the game credits.

The motivation to adapt such a scenario was both to show my love of the original work and also to see how feasible it would be to adapt such a action-adventure focused game to text-based adventured, from my work I believe it is quite adaptable. Such as in the game proper, enemies can dynamically re-spawn and I have tried to the best to my abilities replicate and adapt such a feature, where there is a chance in rooms where no enemy current exists to infect such a room, alert the player and now what once was a safe room becomes crawling with a hiss enemy, leading to in-predicability and randomness to the gameplay look and keeping the player tense when moving into pre-explored rooms/locations.

Since the games setting is a late 2010's new weird surrealist tone, the flavour text from reading notes and reading Jesse's inner thoughts plus the NPC dialogue all share such a tone, always trying to confuse the player in dreamlike logic and hint towards the horrors lurking deeper in the explorable Oldest House.

Knowledge Base Design

Exploration, Puzzles and Storytelling

Locations in the game are internally called room, these room predicates are expanded with static facts, such as description, room name, notes and inner thoughts. Since they are

static they are initialised once at runtime and can not change since I these predicates just define the truth about room location and thus do not need be dynamic since that would increase the performance overhead and overcomplicate the manageability of the prolog source-code.

Navigation therefore is expressed via room connections, these connections are a mix of static and dynamic facts, with most being static however locked rooms being dynamic facts where reasoning is done at query time to determine whether or not the player has solved a puzzle or has the required item to dynamically change the state of the locked room to a unlocked room, this bypasses the need for separate unlock and lock states for applicable room and other locked objects.

Entities, that being the player character, enemies and interactable items are modelled via dynamic single fact per entity representation, for example a pick-up-able item fact can be a has or object status, meaning that if the player is holding the object it uses the has status however if it yet is to be picked up it exists as a object fact located in a room and with a id that the player can typed to pick up said object. This makes sense as the player is either holding or has the object in their inventory or does not thus ensuring has object logic patterns work cleanly and ensures dynamic objects are dealt with strictly thus significantly increasing the robustness of the game as objects cannot exist in two states that could otherwise lead to undesired or duplicate or crashing checking rules. Initially I tried to implement an additional owner attribute to the items predicates, however the additional type-checking performance penalities I deemed not needed as only the player can pick items up and I did not plan for enemies to pick up items as they could possibly pick up puzzle items and that would require for additional checking whether or not a item is essential or a puzzle object, thus the compromise of a item either part of the players inventory or existing in the room was chosen.

Combat and Upgrades

Enemies can be both declared as static facts, such as the boss enemy existing in the penultimate room or dynamically spawned in via the infection system. There are different types of enemies, each with their own name and id that can be used to attack, all enemies require a room to exist in and determining whether or not a entity is hostile is done via a static classifier is hiss enemy check, quickly checking if a entity type is of the list of declared enemies meaning I can differentiate friendly npc's from enemy npc's, where friendly npc's have static dialogue flavour the player can request via the talk command. This design allows me to re-use the entity predicate however intelligently use the game to determine whether or not a NPC is friendly or enemy.

Furthermore, combat status, such as hp is tracked for both the player and enemy entities dynamically, updated after each combat turn, since you can find upgrades in the game, the max health and max weapon charge is additionally expressed as a dynamic fact, and the max values are used as a cap to ensure proper weapon cool down and health regeneration. Designing with the cap paradigms allowed me to easily upgrade health and charge such as adding 5 hp onto the max cap when health kits are picked up and likewise for charge upgrades that add 2 extra charges for the service weapon, constraint by a defined maximum cap also to balance extreme modifications to the player status attributes.

Finally nullary predicates is used for tracking important game states, such as game over and weapon just taken to enable combat or not as the player starts initially unarmed, this was used as it's an extremely quick way of determining whether or not the player has achieved flagged goals. And when a user restarts the game all these facts are reverted to their initial states allowing for the game to be re-experienced from scratch without updated facts ruining and misconstruing the gameplay.

Reasoning and Rules

Win and Loss Conditions

Both the Win condition, finding and talking to the Dylan NPC and the Loss Conditions when hp reaches 0 or below are verified through atomic checks, if any of these two states are met then corresponding nullary predicate flags are raised asserting the game over flag, over in the case it was completed and won and over in the case the player died with differing output text accordingly, the you died message or the game credits, both prompting the user to restart or quit the game. I could have used two different trackers to track said win and loss conditions however just using one is more efficient as both conditions are handled by the same flag assertion so why use 2 rules to do the same job that can be done in 1 rule.

Inventory Management

Aforementioned objects are managed by the inventory through the mutually exclusive relationship of the two predicates has and object, where has is asserted when a player picks up a object and that object is retracted from the room world and any initial effect is applied for example restoring health or enabling combat, this ensures an object is either interactable in the room or in the players inventory from where the player can invoke inventory and see the entire contents of their inventory, and use the place predicate to return the item to object state within a room, required for certain puzzle tracking if object is in room or not. This also ensures items cannot be picked up twice as they are finite and should not be duplicable by the player, for example this method prevents the player from duplicating health kits that would make combat unchallenging and unfair.

Actions

Movement and combat is heavily constraint and gated by action predicates, for example for the attack action to apply the game first verifies if the player has the service weapon and if the enemy is alive and in the same room, if not the action will not succeed. Moving into locked rooms will only succeed if the player has meet the requirements for example having a key in their inventory and if not the movement will be denied and a message to the user telling them the action is invalid and how to overcome such inaction. This ensures gameplay is lazily evaluated by prolog in room space, thus not requiring expensive searches into other connections, keeping both

combat and movement local to one room meaning less unnecessary states to track, increasing performance and reliability of core gameplay rules.

States

Updating information is handled safely through use of helper predicates that retract the old facts such as current health and charge and assert the new values meaning there will be no such scenario where a player has two health facts or charge facts which particularly enhances the flow of combat, reducing scenarios where enemy health deltas are incorrectly or unordered thus making the combat safer and less prone to state update bugs.

Utilities

The prolog built in random predicate allows for the infection system to have a 10 percent chance each movement step to spawn a enemy in a enemy-less room allowing for un-deterministic gameplay and also allowing for the chance of critical hits during combat and the ten percent chance of a weapon misfire making combat less predictable and more engaging, these rules are wrapped in multi-clause dispatches which can dynamically update combat flavour text to account for the critical hit acknowledgement rather than using less efficient nesting within if else conditioning and pre-calculation of randomised infused facts preventing the likelihood of backtracking. This also keeps my rules organised and logically next to one another not only enhancing the maintainability of the source code but keeping the pattern matching optimal for the performance of the prolog compiler.

Example

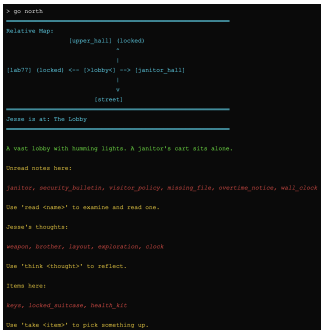


Figure 1: Interactable objects; notes, thoughts and items plus the always avaliable dynamic map.

Above, the objects in red can be easily identified as being Interactable, through the use of read, think and take. Furthermore, the relative map of the current location is always avaliable to the player, showing firstly when entering a new location or through use of the map command, the map and interactable contents serve as the visuals of the game, what the in-game protagonist Jesse sees and informs what she can do and where she can go from here, thereby ensuring the player can navigate the game world and get a sense of what to do in the game world, without this visual information the game would not be playable and as you can see my colourisation, stylisation was carefully considered to ensure players could readily distinguish the parts of the text adventure and clearly see dynamically which areas are currently locked to the player and make their informed adventure choice.

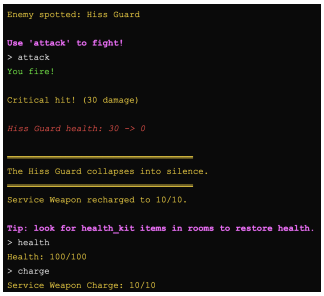


Figure 2: An example combat scenario, a critical hit and player status information and flavour text during combat.

Note, the player is firstly alerted to the presence of a enemy, then proceeds to score a random critical hit, one that defeats the enemy instantly and drops a flavour text indicating the enemy is now dead, for balancing reasons the weapon is fully recharged so-to if the player encounters a enemy in the next room they have a fighting chance without spamming skip, else-more each combat turn clearly shows the enemies health and the players current health from the enemies attack as-well as the weapon charge to inform the player if they need to skip or shield or throw et-cetera to balance their fluctuating resources.

Conclusion

Due to time constraints, I hastily adapted the story of control, missing many major-plot points and cutting characters, however the goal of finding the brother I deemed to be a workable showcase spirit of the story at-large, hence If I had more time I would fundamentally adapt the complete story, plus adding additional unlockable abilities, weapon forms, enemy types and locations expanding and enhancing the runtime of the game and making it feel less monotonous, all of this currently can be expanded with the existing knowledge-base facts as the encapsulating logic is already there, I would only just need to expand the content using said clauses, adding more note and thought text and increasing the range of dialogue from the NPC's and adding in the ones that fell in the cutting- room floor.

Additionally, connecting the current prolog logic to a game-engine such as Godot or Unity could allow me to introduce primitive and more-interesting 2D or 3D graphics greatly enhancing the presentation and user-experience of the game. However as the game currently stands, I believe it to be a great showcase of how a text-adventure game in prolog could be designed and implemented, with a robust and extensive knowledge-base and rule-set allowing for gripping, error-free gameplay that has game-over states and a definitive desirable end-state goal to complete the game.